

Tema 2:

Sistemas distribuidos tipo cluster

Sistemas Distribuidos

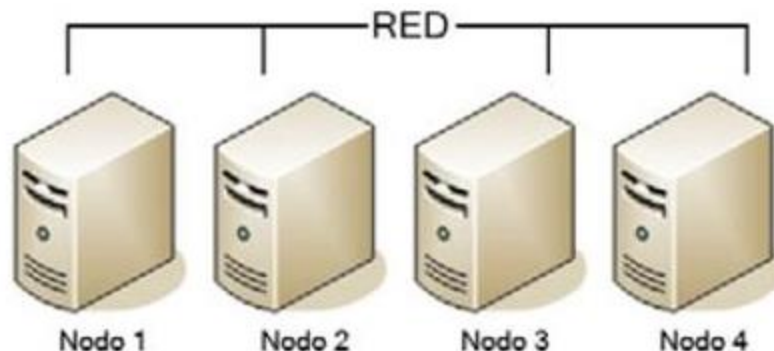
Marcos Novalbos
Marcos.novalbos@u-tad.com

Índice

- **Introducción**
- Computación Cliente/Servidor
- Llamadas a procedimientos remotos
 - Paso de mensajes
 - RPC
- Clusters
- Gestión de procesos distribuidos
 - Migración de procesos
 - Exclusión mutua e interbloqueo

Introducción

- Un sistema de memoria distribuida se compone de un conjunto de nodos de cómputo completos e independientes, conectados entre sí por una red de comunicaciones.
- Este tipo de sistemas se suelen denominar “multicomputador”:
 - Su rendimiento va ligado al programa de gestión o S.O. distribuido que se use para su explotación



Introducción

- Para poder usar estos sistemas se suelen usar estrategias de “procesamiento de datos distribuido” (DDP).
 - Los computadores se encuentran dispersos en un área, interconectados por una red.
 - Su objetivo es procesar la información de la forma más eficiente posible, aprovechando las capacidades de procesamiento paralelo y replicación de unidades.
 - No todos los nodos son iguales, se elegirán nodos en función de sus características.
 - Se deben añadir funciones de recuperación ante errores, pérdidas de comunicaciones, caídas de nodos, pérdidas de datos...

Introducción

- Características de un DDP:
 - **Disponibilidad:** Con múltiples sistemas interconectados, una unidad replicada podría asumir la carga de la pérdida de uno de ellos rápidamente.
 - **Compartición de recursos:** El hardware/ software más caro se podría compartir entre los usuarios. Las bases de datos se pueden mantener de forma distribuida entre las instalaciones.

Introducción

- Características de un DDP:
 - **Crecimiento incremental:** No hace falta tirar los equipos viejos al comprar nuevos. Se pueden añadir sistemas nuevos para aplicaciones con mayor carga de procesamiento de datos, manteniendo los antiguos para aplicaciones más sencillas.
 - **Mayor productividad:** Los sistemas distribuidos suelen tener una mayor rapidez de respuesta ante programas con gran carga de trabajo, ya que se tiende a que cada nodo de cómputo realice partes pequeñas del mismo (“divide y vencerás”).

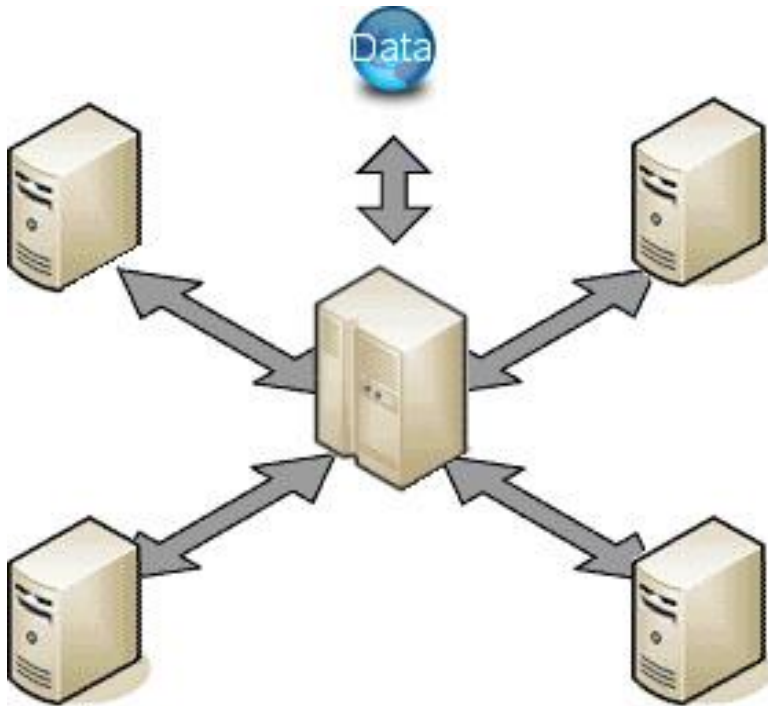
Introducción

- Debido a su naturaleza, se proponen nuevos paradigmas de programación:
 - No hay un único rango de direcciones de memoria
 - Comunicación de procesos por “paso de mensajes”
 - Gestión de recursos
 - Cada nodo puede tener capacidades de autogestión
 - Deben sincronizarse cuando se comparten recursos
 - Esquema Maestro/Esclavo
 - Interfaces de usuario
 - Esquema Cliente/Servidor

Introducción

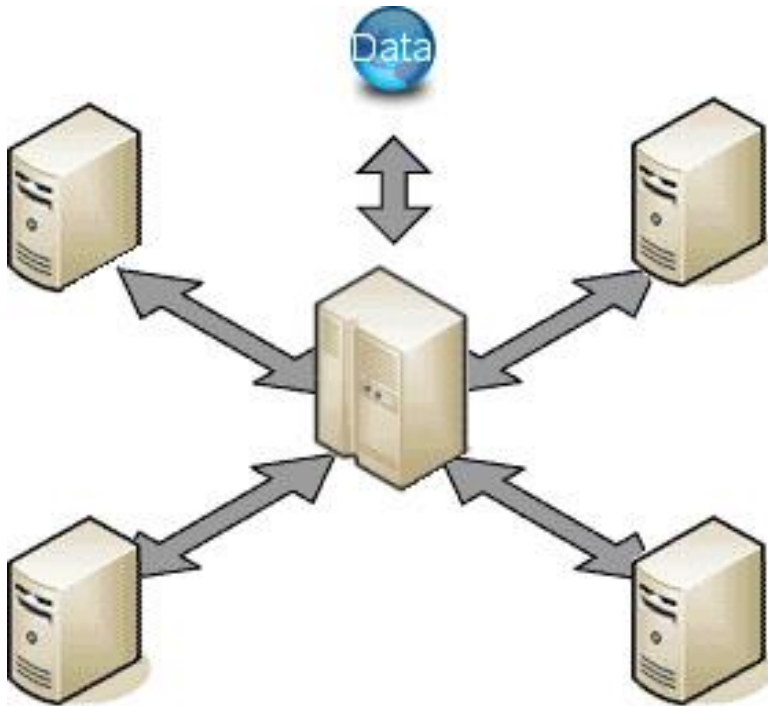
- Para lograr estos beneficios, el sistema operativo debe proporcionar diversas funciones de soporte.
 - Software para intercambio de datos
 - Capacidad para lograr alta disponibilidad y prestaciones
 - Capacidad de gestión de procesos en entornos distribuidos
- Además, la seguridad supone un punto mucho más importante.
 - Protección de datos y recursos privados
 - Garantía de la autenticidad de los usuarios
 - Implementación de diversos sistemas de criptografía y seguridad para conexiones e intercambio de información

Computación cliente/servidor



- Encontramos varias partes:
 - **Clientes:** Normalmente son simples terminales o estaciones de trabajo que proporcionan una interfaz de fácil manejo para el usuario final
 - **Servidor:** Este sistema es más complejo y ofrece los servicios compartidos entre los clientes.

Computación cliente/servidor



- Encontramos varias partes:
 - **Interfaz de programación (API):** Conjunto de funciones y programas que permiten a los clientes y servidores comunicarse.
 - **Middleware:** Conjunto de controladores, API y software adicional que facilitan uso dando una visión global del sistema al cliente.

Computación cliente/servidor

- Clases de aplicaciones cliente/servidor
 - **Procesamiento tipo host:** Toda la carga se encuentra en el servidor, el cliente no es más que un “terminal tonto”.
 - **Procesador tipo servidor:** El cliente proporciona una interfaz gráfica con las opciones a realizar al usuario, y mediante mensajes al servidor éste realiza las operaciones.
 - Usado normalmente en páginas web y bases de datos.

Computación cliente/servidor

- Clases de aplicaciones cliente/servidor
 - **Procesamiento basado en cliente:** En este caso, casi toda la carga de proceso recae en el cliente. El servidor se usa para tareas de validación de datos, pero es el terminal del cliente el que realiza las operaciones.
 - Ej: Juegos en red
 - **Procesamiento cooperativo:** En este caso, tanto los clientes como el servidor realizan parte del trabajo, repartiéndolo de forma equilibrada entre las estaciones.
 - Ej: Una base de datos con un sistema de ficheros distribuido entre los clientes

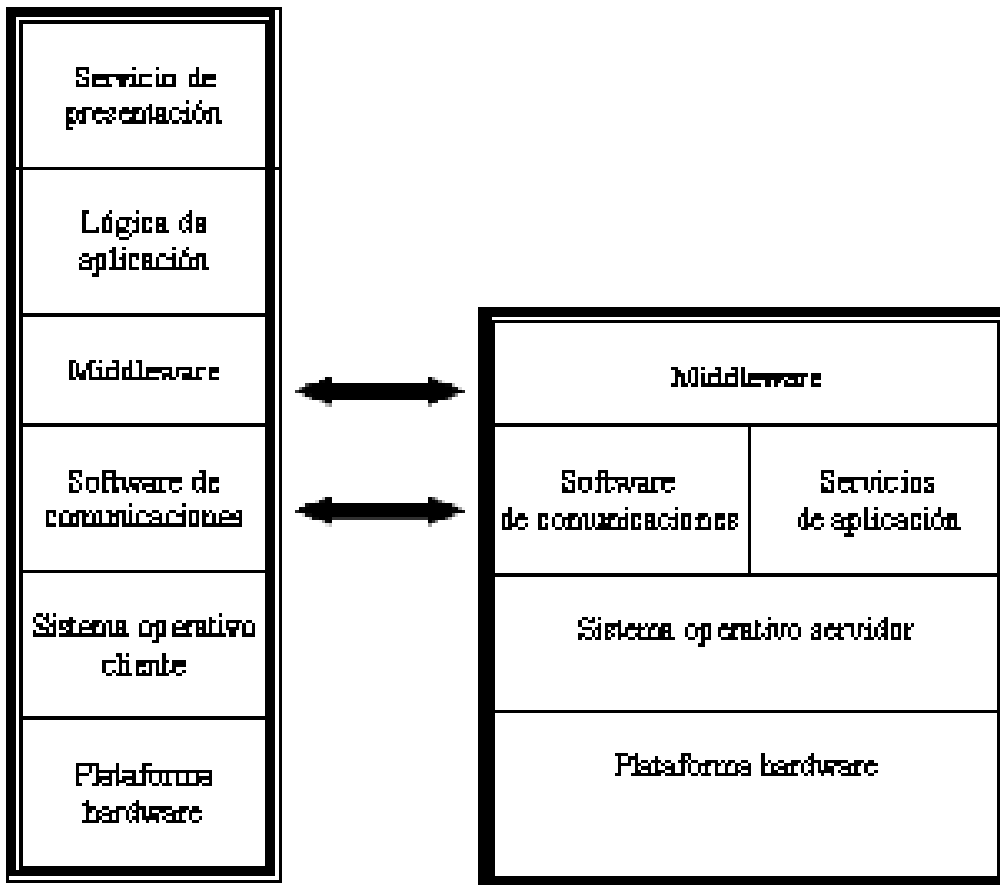
Computación cliente/servidor

■ Middleware

- La evolución de los sistemas cliente/servidor ha desembocado en una gran dificultad para estandarizar todos los aspectos de la computación distribuida.
- Para poder sacar provecho de estos sistemas, los desarrolladores necesitan unas herramientas que den una visión uniforme a través de todas las plataformas.
- Aplicaciones similares en todos los PC
- Misma forma de acceso a datos, independientemente de dónde se encuentren.

Middleware

- Para poder cumplir con esos requisitos se suelen usar interfaces de programación, que se encuentran entre la aplicación, el software de comunicaciones y el sistema operativo, denominados “Middleware” (software intermedio).



Llamadas y gestión de procedimientos remotos

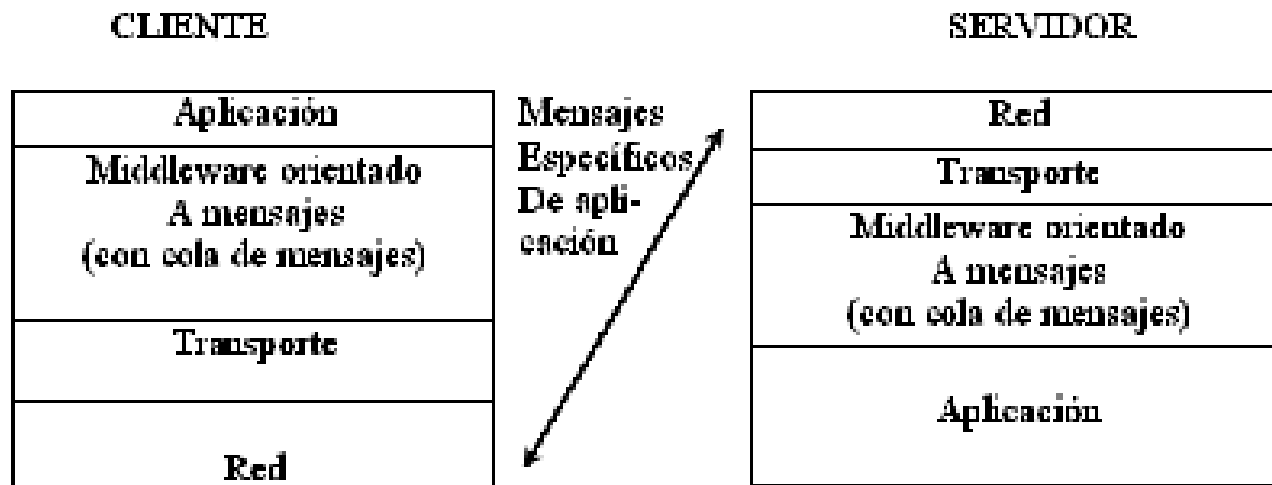
■ Paso de mensajes

- En los sistemas de procesamiento distribuido, para comunicar y sincronizar procesos se usan técnicas basadas de paso de mensajes.
- Se definen una serie de funciones:
 - Send: Mandar mensajes, con los parámetros necesarios para ejecutar la petición y un identificador de nodo. Opcionalmente se puede enviar a todos.
 - Receive: Recepción del mensaje emitido, normalmente a través de un buffer de recepción. Se indica además un identificador de nodo del que se quiere recibir, opcionalmente se puede recibir de todos.

Llamadas y gestión de procedimientos remotos

■ Paso de mensajes

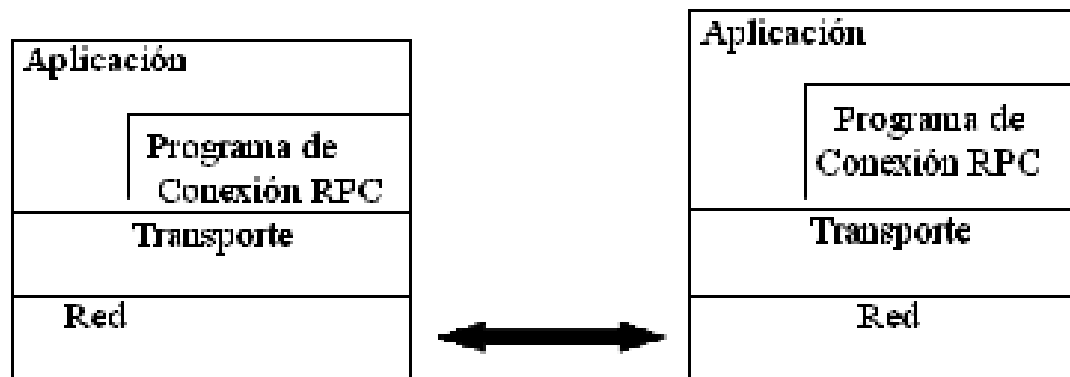
- **Middleware orientado a mensajes:** Una aplicación usa una interfaz de programación para ponerse en contacto con otra. El middleware le permite usar funciones conocidas de Send y Receive.



Llamadas y gestión de procedimientos remotos

■ Paso de mensajes

- **Llamadas a procedimiento remoto:** La interfaz es una función normal, que abstrae de las funciones Send y Receive al programador. La invocación implementará por debajo esas funciones, y ejecutará el trabajo en la estación remota.

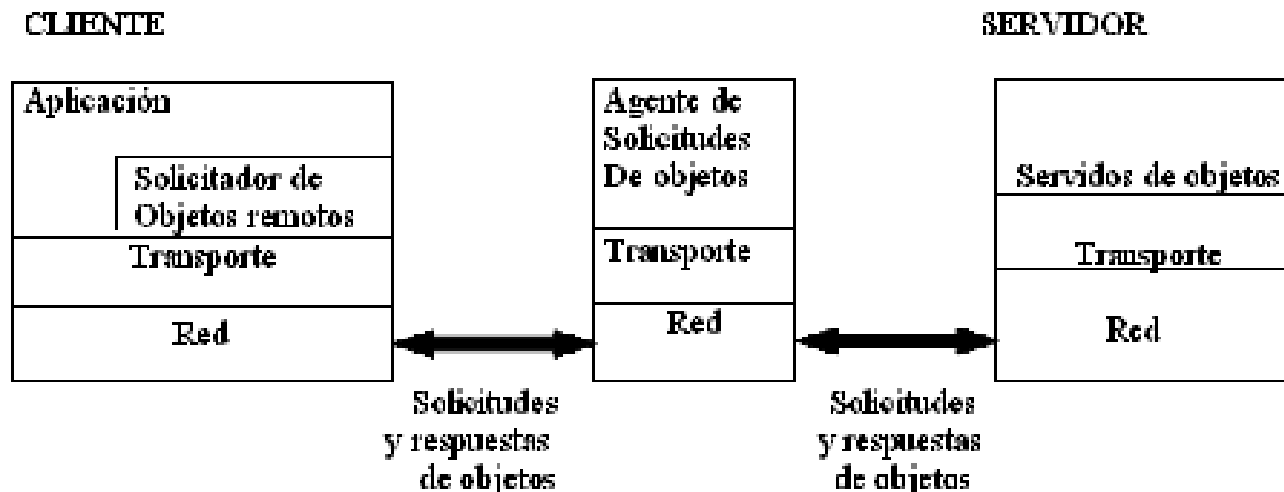


Invocaciones y retorno de
Procedimientos específicos
De la aplicación.

Llamadas y gestión de procedimientos remotos

■ Paso de mensajes

- **Broker de objetos:** Implementación de RPC sobre objetos. Cuando un cliente necesita un servicio, manda una petición a mediador actúa como directorio de servicios remotos. Éste le pone en contacto con el nodo que contiene el objeto adecuado.
- Ej: Modelos COM de Microsoft, CORBA...



Llamadas y gestión de procedimientos remotos

- Las funciones de pasos de mensajes pueden ser:
 - **Fiable/No fiable:**
 - Los sistemas fiables garantizan la entrega cuando es posible. Usan sistemas de transporte fiable, con control de errores, acuse de recibo transmisión y reordenamiento de mensajes desordenados.
 - Los no fiables no lo garantizan. Sin embargo, reducen mucho la complejidad y la sobrecarga de información por envío extra de datos. Sería la propia aplicación la que implemente un protocolo de fiabilidad para asegurar la entrega de información.

Llamadas y gestión de procedimientos remotos

- Las funciones de pasos de mensajes pueden ser:
 - **Bloqueante/No bloqueante:**
 - Las llamadas bloqueantes paran la función llamante de send/receive hasta que el mensaje ha sido enviado.
 - Las “no bloqueantes” devuelven el control a la función tan pronto como sea posible (el mensaje ha sido puesto en la cola de envío). Normalmente se usa una variable que indica el estado de la llamada (terminada/no terminada), y un buffer de datos de envío/recepción. Mientras que la variable no indique que la llamada ha terminado, no se pueden reutilizar los búferes de datos.

Llamadas y gestión de procedimientos remotos

- Las funciones de pasos de mensajes pueden ser:
 - **Bloqueante/No bloqueante:**
 - Las llamadas no bloqueantes son muy importantes para solapar los tiempos de comunicación
 - La red es muy lenta, mientras se prepara el mensaje, se envía, se ejecuta en el servidor, y se devuelve el resultado, el cliente permanece parado.
 - Con un envío no bloqueante se obtiene un “recibo”, que se consultará cuando se requiera el resultado de la llamada anterior. Mientras que ese recibo no indique que tenemos respuesta, realizamos trabajo útil.

Llamadas y gestión de procedimientos remotos

■ Llamadas a procedimientos remotos

- Las llamadas a procedimientos remotos (RPC) son una variante del modelo de paso de mensajes.
- Encapsulan la comunicación en un sistema distribuido, permitiendo a las aplicaciones abstraerse del entorno distribuido.
- A ojos del programador, son llamadas a funciones normales, no tiene por qué conocer la infraestructura donde se tendrá que ejecutar la aplicación.

Llamadas y gestión de procedimientos remotos

- Proporcionan las siguientes ventajas:
 - Las RPC permiten especificar las interfaces remotas como conjunto de operaciones con nombres y tipos de datos dados. Así se pueden documentar y comprobar errores en los datos recibidos.
 - Una vez especificada una interfaz, el código de comunicación se puede generar automáticamente.
 - A partir de la interfaz, se pueden escribir módulos cliente/servidor portables entre arquitecturas. Solo es necesario reimplementar la funcionalidad exportada en la interfaz.

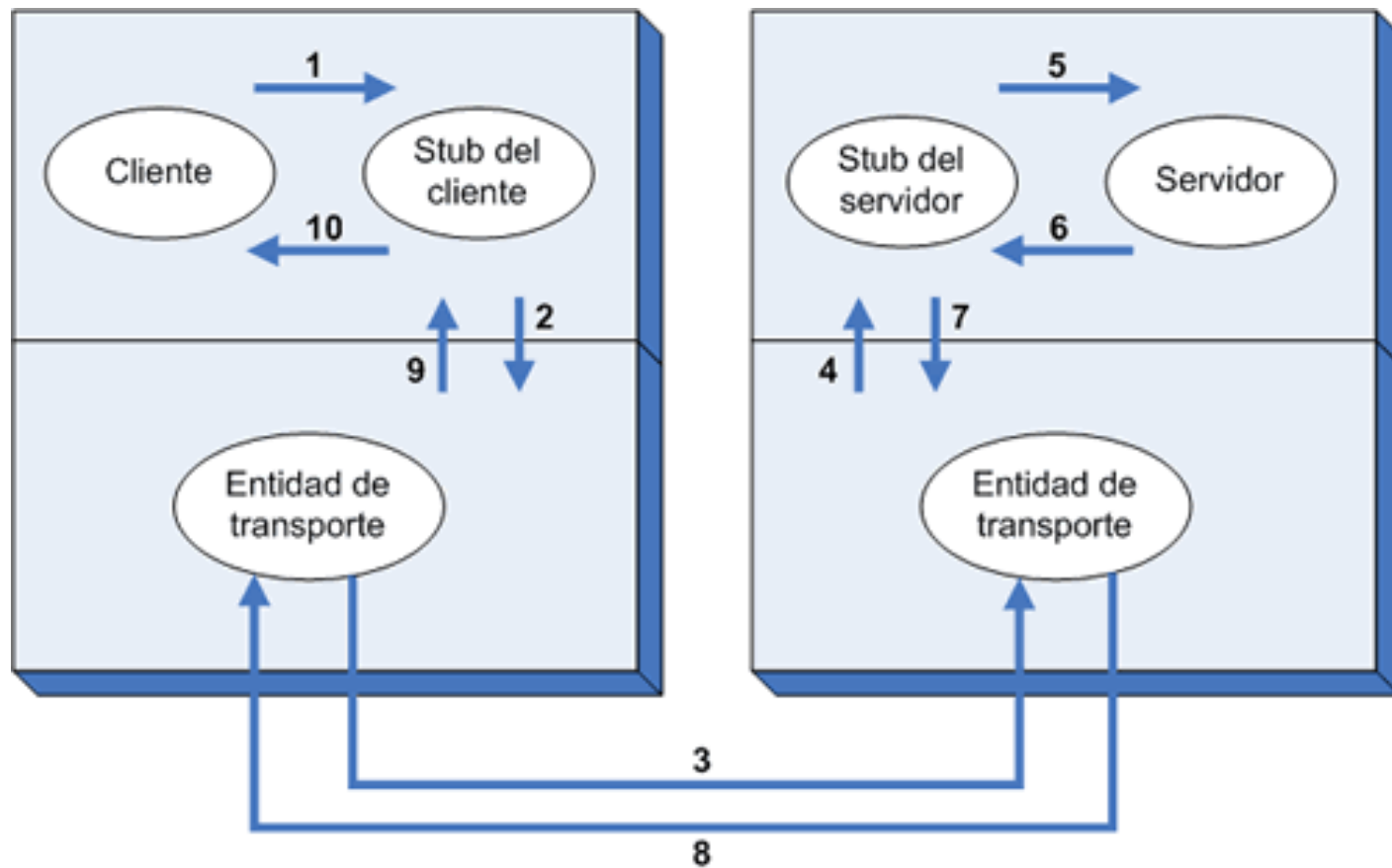
Llamadas y gestión de procedimientos remotos

- El proceso de ejecución suele ser el siguiente:
 - El programa realiza una llamada a un procedimiento normal de la máquina
 - Ej: CALL P(X,Y)
 - P= nombre de procedimiento
 - X= argumentos de entrada
 - Y= argumentos de salida
 - Normalmente, en el espacio de direcciones del llamante se incluye un “esqueleto” (stub), que se enlazará en tiempo de llamada.
 - Estos esqueletos incluyen un sistema de comunicación que identificará el sistema remoto, y enviará la petición de ejecución con los parámetros necesarios.

Llamadas y gestión de procedimientos remotos

- En la máquina remota hay otro programa esqueleto asociado al procedimiento al que se ha llamado.
- Al llegar un mensaje de “CALL P(X,Y)”, se examina y se ejecuta de forma local. En este caso se dispone de una implementación local de la llamada, por lo que su ejecución se correspondería a una normal, con sus estados de pila y parámetros.

Llamadas y gestión de procedimientos remotos



Llamadas y gestión de procedimientos remotos

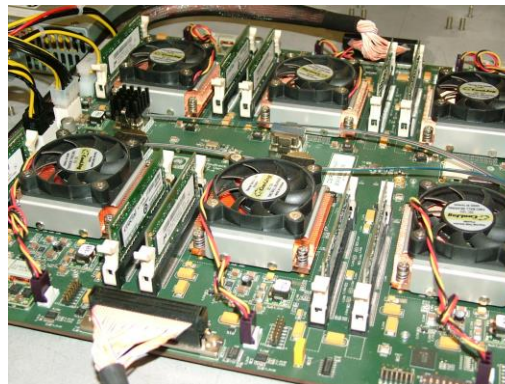
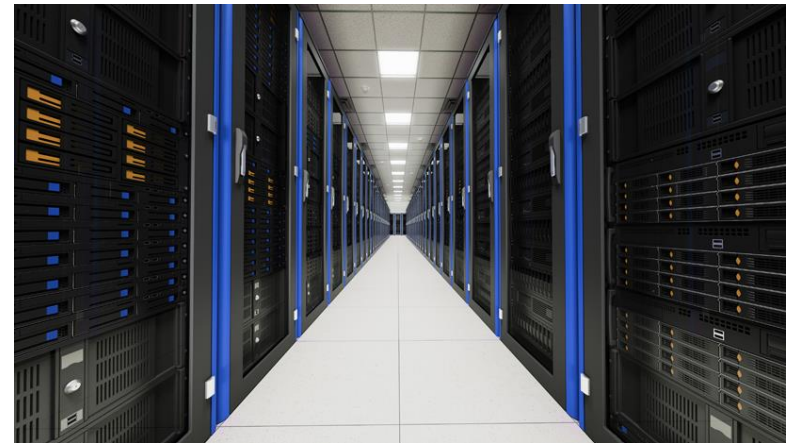
- **Representación de parámetros**
- Un punto importante de los RPC es que los programas cliente y servidor no tienen por qué estar escritos en el mismo lenguaje ni arquitectura.
- Algunas inconsistencias se pueden solucionar a nivel de envío por red, pero si no hay un protocolo no se puede asegurar que lo enviado y recibido a ambos lados sea interpretado igual.
- Algunas herramientas de RPC proporcionan métodos de estandarización para los tipos de datos comunes (Caracteres, floats, enteros...), y funciones para crear registros y estructuras de datos complejas.
- De esta manera, los parámetros nativos de cada máquina se pueden convertir a una representación estándar.
 - Más trabajo a la hora de enviar/recibir datos.
 - Facilidad de manejo de datos entre distintos entornos.

Clusters

- Este tipo de arquitecturas han aparecido de manera natural en pequeñas empresas, universidades y centros de investigación debido a:
 - Grandes avances en las prestaciones de los procesadores de consumo habitual
 - Aparición de redes de comunicación y protocolos de alto rendimiento
 - Desarrollo de herramientas software que facilitan su configuración y administración.

Clusters

- Se pueden distinguir entre distintos tipos de clusters:
 - Dedicados/no dedicados
 - Homogéneos/Heterogéneos
 - Propietarios/Beowulf
- En cualquier caso, las aplicaciones más comunes son:
 - Internet
 - Bases de datos
 - Cálculo científico.



Clusters

- Caso de estudio: Pixar
 - Toy Story, 1995
 - 15 horas en renderizar cada frame
 - 114.000 frames, 77 minutos de película
 - Un único equipo (de la época) habría tardado 43 años
 - Cluster (Granja de render):
 - Inicialmente 53 nodos. Tardó 20 meses
 - Creció hasta los 300 nodos
 - Pixar cuenta en la actualidad con 23.000 procesadores



¡Suficiente para renderizar la película original en tiempo real!

Clusters

- Siempre existe un problema más grande
 - “Transformers” necesitó 38 horas para renderizar cada fotograma de la animación
 - “Los increíbles” necesitó 69 horas para renderizar cada frame



Clusters

- Simulación de fluidos. “Battleship”



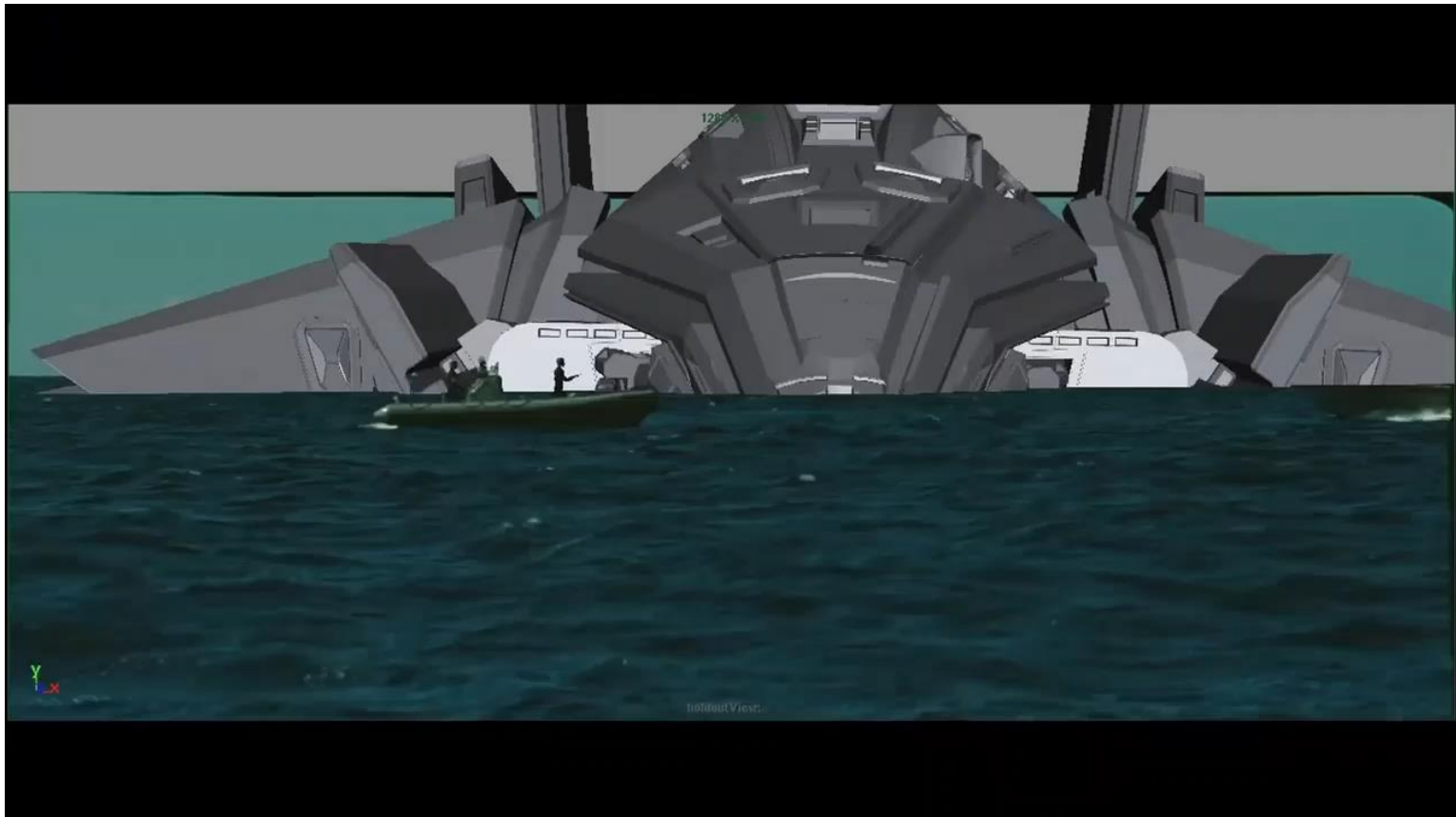
Clusters

- Simulación de fluidos. “Battleship”



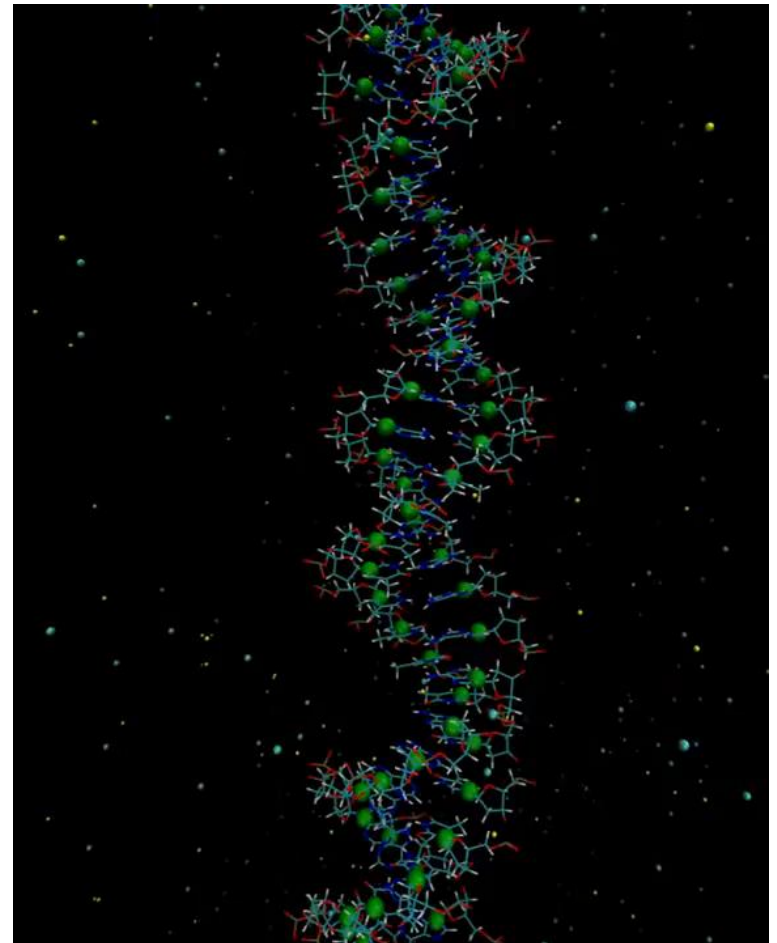
Clusters

- Simulación de fluidos. “Battleship”



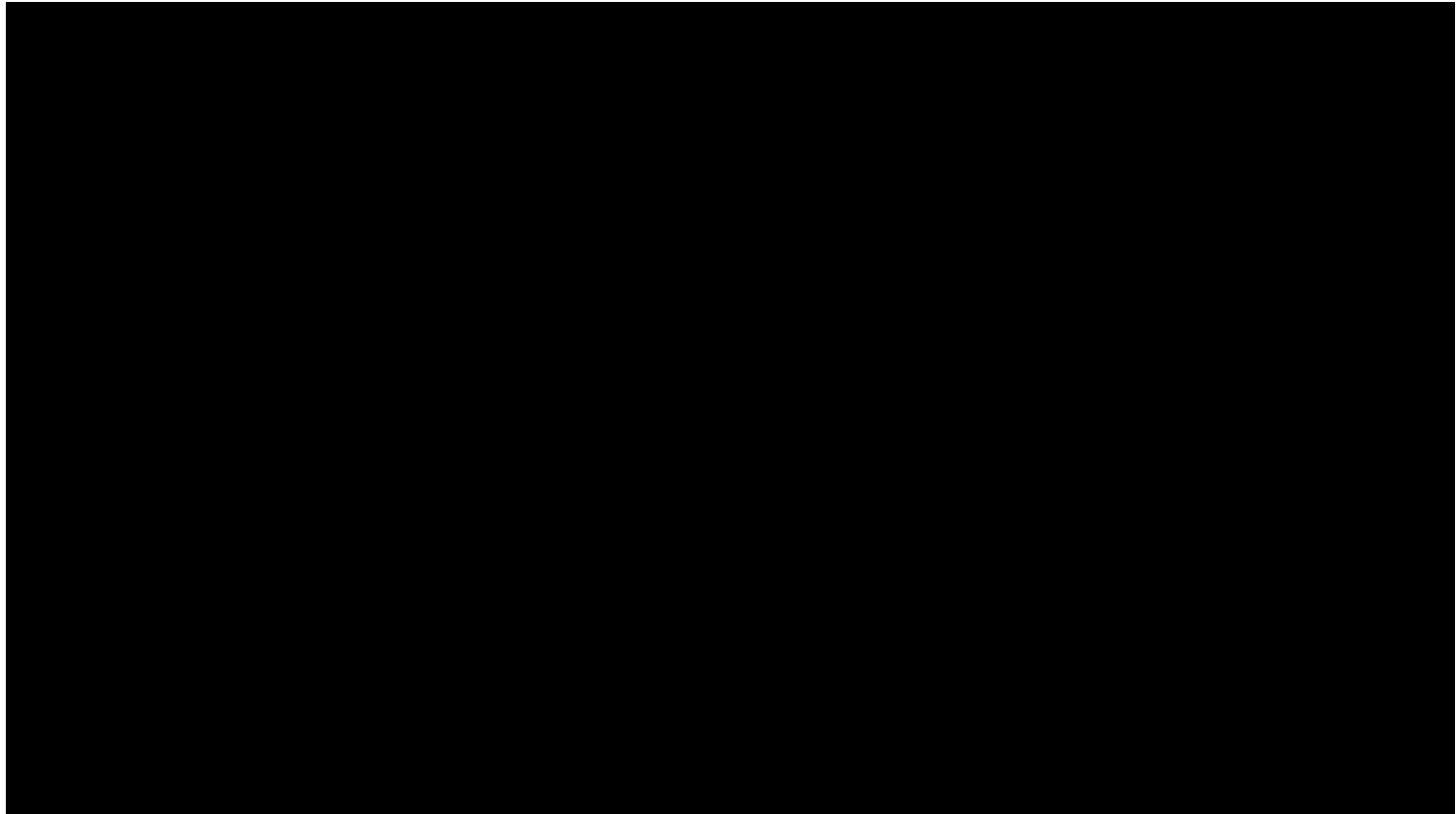
Clusters

- Simulación de dinámica molecular



Clusters

- Simulación de dinámica molecular



Clusters

- La simulación de dinámica molecular supone tratar con una gran cantidad de datos para obtener resultados muy pequeños
 - Cada paso de simulación supone un paso de tiempo de 1 fs (milbillonésima parte de 1 s)
 - Un grano de arena se compone de 2,2 trillones de átomos
 - Almacenar 2,2 trillones de posiciones supone un uso de mas de **24 millones de Terabytes** (24.586.915.970 GigaBytes)
 - Es un proceso largo y costoso
 - Se necesita emplear un (o varios) supercomputador (cluster)

Clusters

- Aplicaciones prácticas.
Ciberseguridad
 - Congreso BlackHat 2011
 - Se presenta un software que de atacar por fuerza bruta claves WPA2 en servicios de cloud computing
 - En el cloud computing de Amazon es capaz de “romper” la contraseña WPA2 en menos de 20 minutos. En el mismo congreso se optimiza y reduce a 6 minutos
 - 8 equipos con: 22GB RAM, 2 Intel Xeon X5570, 2 NVIDIA Tesla M2050.
 - Congreso BlackHat 2012
 - Se presenta otro software, capaz de “romper” la seguridad de los protocolos MS-CHAPv2(EAP/PEAP) en servicios de cloud computing



Clusters

- Se pueden tener varias configuraciones para el cluster.
 - La más sencilla sería una serie de ordenadores conectados en red de alta velocidad (infiniband)
 - La más común es tener al menos un sistema de ficheros compartido de alguna forma:
 - Un nodo con el sistema de ficheros para todos
 - Un sistema de ficheros distribuido con RAID.

Clusters

- Aspectos de diseño de SO para cluster
 - **Gestión de fallos:** se suelen seguir dos enfoques:
 - **Alta disponibilidad:** Ofrece la posibilidad de que todos los procesos estén en servicio. Si se pierde un nodo, se pierden las peticiones que hubiera en el.
 - **Tolerancia a fallos:** Asegura que todos los procesos están disponibles siempre, gracias a redundancia de unidades y mecanismos de replicación/migración de procesos

Clusters

- Aspectos de diseño de SO para cluster
 - Equilibrado de carga:
 - Un cluster necesita equilibrar las cargas de trabajo entre los nodos de cómputo, trabajo que debería de tratar el S.O. Además, al añadir nuevos nodos, éstos deberían de ser incluidos en el planificador de procesos.
 - Los mecanismos de Middleware que implemente el SO deberían de ser capaces de reconocer nuevos servicios en diferentes miembros del cluster, para permitir migrar servicios a ellos.

Clusters

- Aspectos de diseño de SO para cluster
 - Computación paralela:
 - **Compilación paralela:** Un compilador paralelo puede determinar, en tiempo de ejecución, que partes de la aplicación pueden ejecutarse en paralelo. Éstas partes se pueden asignar a distintas partes del cluster para ejecutarse en paralelo y comunicarse mediante paso de mensajes.

Clusters

- Aspectos de diseño de SO para cluster
 - Computación paralela:
 - **Aplicaciones paralelas:** Aquí es el programador el que escribe la aplicación específica para el cluster. Usa una API de paso de mensajes para mover datos entre máquinas y aprovechar los recursos
 - Es un gran trabajo para el programador, pero puede ser el mejor enfoque.

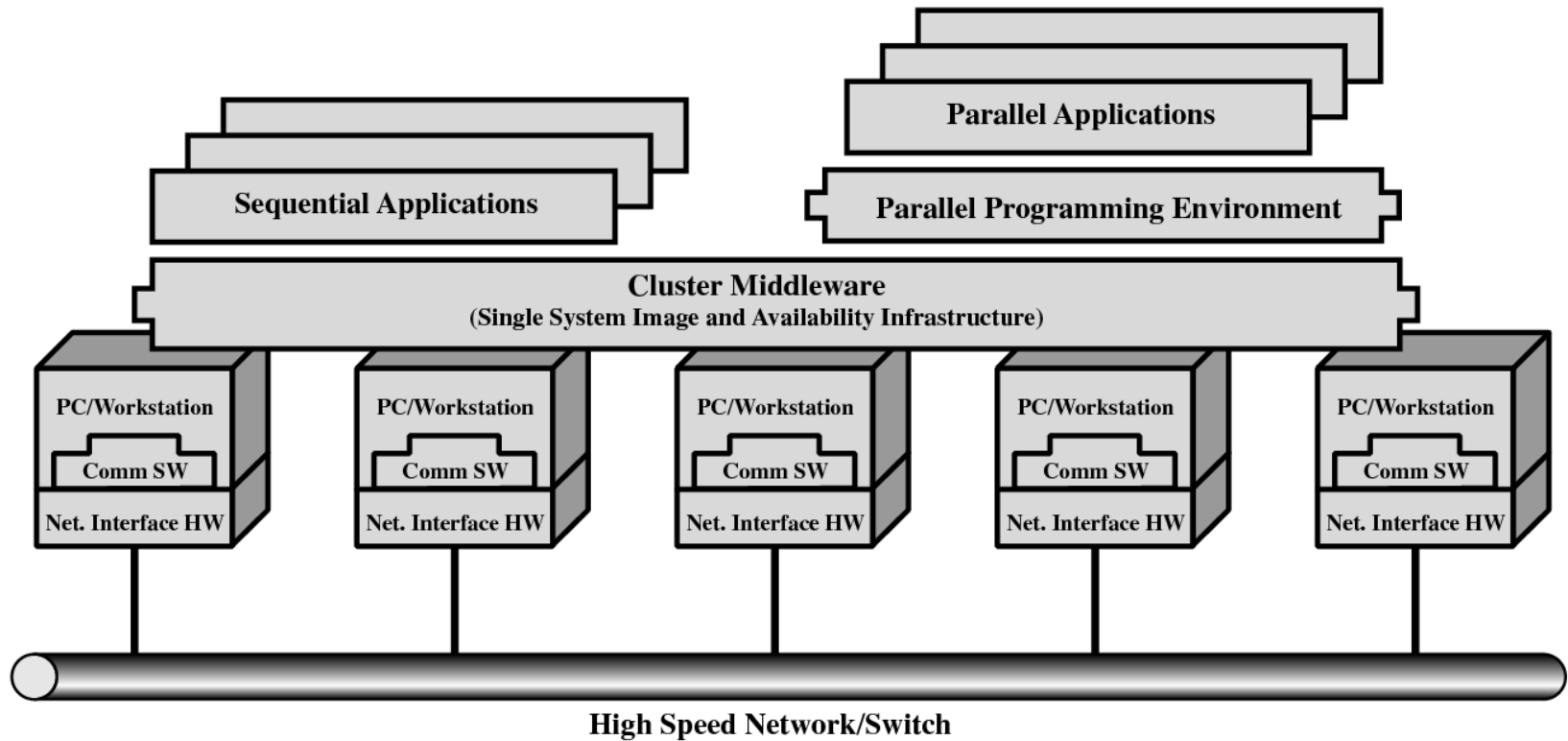


Figure 18.11 Cluster Computer Architecture [BUY99a]

Gestión de procesos distribuidos

■ Migración de procesos

- En entornos distribuidos es importante que las cargas de trabajo se encuentren repartidas de forma equitativa.
- Aparece una necesidad de mecanismos para migrar procesos de nodos más cargados a menos cargados.
 - Definición: Transferencia de suficiente cantidad del estado de un proceso de un computador a otro, para poder ejecutarse.

Gestión de procesos distribuidos

- Algunos aspectos a tener en cuenta:
 - **Compartición de carga:** Migrar procesos de nodos cargados a los poco cargados
 - **Rendimiento de las comunicaciones:** Si hay procesos que interactúen mucho, sería conveniente colocarlos en el mismo nodo
 - **Disponibilidad:** Mover procesos de “alta durabilidad” de nodos en trámite de apagado a nodos que permanezcan encendidos.
 - **Uso de facilidades especiales:** Mover procesos a nodos donde haya recursos específicos que sean necesarios.

Gestión de procesos distribuidos

- **Mecanismos de migración de procesos**
- Antes de migrar un proceso, hay que tener en cuenta ciertos aspectos:
 - ¿Quién inicia la migración?
 - Módulos del sistema operativo, el propio proceso...
 - ¿Qué parte del proceso se migra?
 - Todo el proceso, variables, bloques de control de proceso, ficheros abiertos, mensajes...
 - ¿Qué sucede con los mensajes y señales pendientes?
 - Redireccionado desde el nodo antiguo o avisar de la nueva dirección

Gestión de procesos distribuidos

- ¿Quién migra el proceso?
 - Si el objetivo es el equilibrado, será un módulo del sistema operativo o “monitor” que decida cuando debe migrarse el proceso.
 - El módulo local al nodo necesita estar en comunicación con otros módulos para monitorizar la carga de trabajo en el entorno distribuido.
 - Si el objetivo es alcanzar recursos en nodos particulares, la misma aplicación podría migrar el proceso a ese nodo.
 - En este caso, es la aplicación la que debería conocer el estado del sistema y ser “consciente” de los recursos disponibles.

Gestión de procesos distribuidos

- ¿Qué se migra?
 - Normalmente, se destruye el proceso en el sistema origen, y se recrea en el sistema destino. Como mínimo, se mueve su bloque de control de proceso.
 - Los procesos pueden tener enlaces abiertos con otros procesos. Habrá que redireccionar mensajes al nuevo nodo.
 - La dificultad del movimiento de procesos se encuentra en los datos dentro de su espacio de direcciones, y los ficheros abiertos. Suponiendo que se encuentra en un sistema con memoria virtual (paginación o paginación/segmentación), se pueden considerar distintas alternativas.

Gestión de procesos distribuidos

- **Ambicioso (completo):** Se transfiere todo el bloque de memoria en el momento de la migración. En caso de que sea muy grande y el proceso no necesite de toda la memoria, sería una pérdida de tiempo (en orden de minutos quizás).
 - Puede ser muy costoso en tiempo.
 - Es el mejor enfoque, ya que no debería quedar rastro en la máquina origen del proceso.
 - Usado en implementaciones con puntos de recuperación, donde se para un proceso y se guarda su estado con toda la memoria.

Gestión de procesos distribuidos

- **Precopia:** En este caso, el proceso continúa ejecutándose mientras la memoria se copia al nodo destino. Si hay modificaciones en memoria mientras se envían los datos, habrá que volver a copiar las modificaciones.
 - Similar al caso anterior, no queda rastro del proceso en el nodo origen.
 - Reduce el tiempo que pasa el proceso parado mientras se prepara su migración.

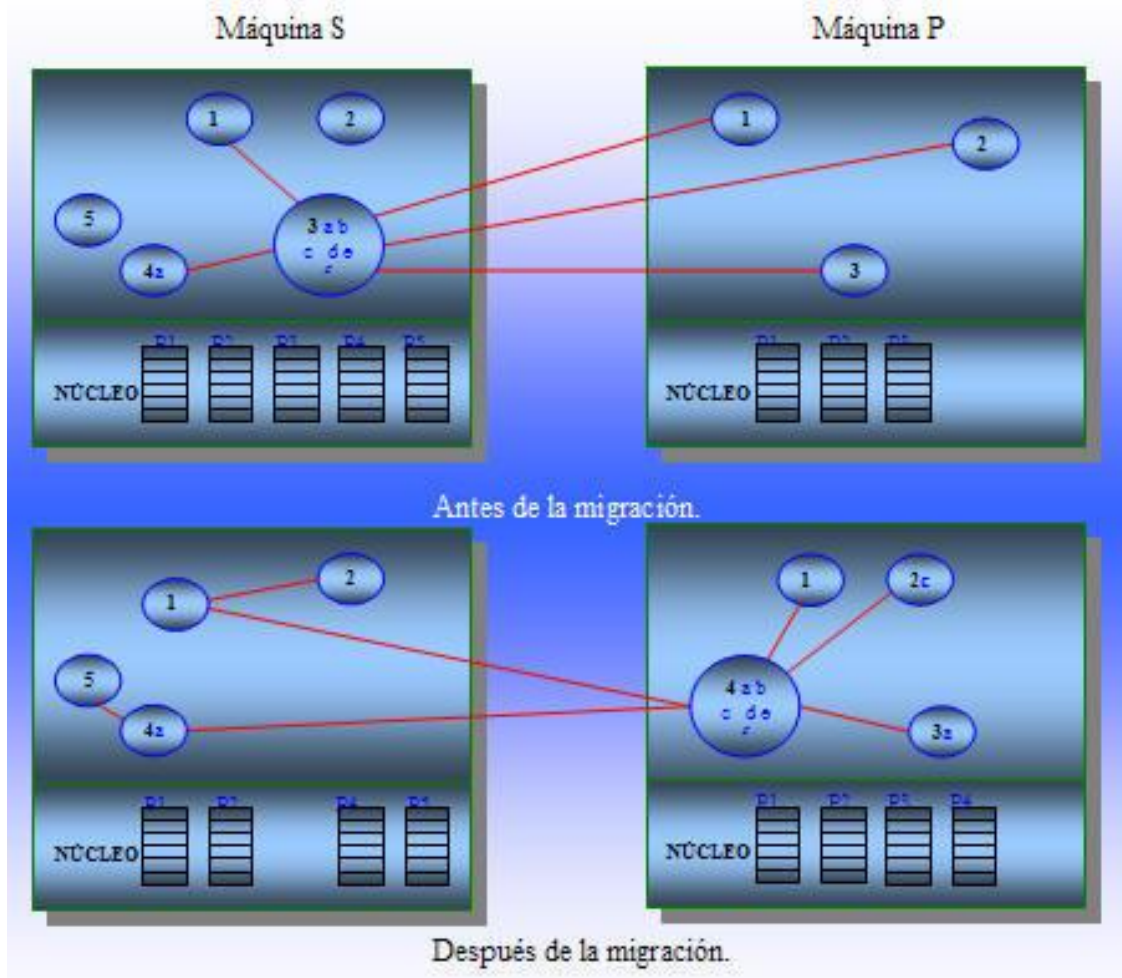
Gestión de procesos distribuidos

- **Ambicioso (no completo):** En este caso, solo se envían las páginas de memoria que se encuentren en memoria principal, y ha sido referenciadas o modificadas recientemente. El resto de páginas de la memoria del proceso se transferirán bajo demanda.
 - Minimiza los datos que se transfieren.
 - Deja restos del proceso en la máquina origen, que deberá mantener las páginas de la memoria del proceso hasta que éste muera o sean copiadas.

Gestión de procesos distribuidos

- **Copiar al referenciar:** En este caso, todas las páginas de memoria se migran bajo demanda. Tiene un menor coste inicial, ya que solo se mueven los bloques indispensables para empezar a ejecutar el programa.
- **Volcado:** Cuando se dispone de un sistema de ficheros compartido por todos los nodos, se puede volcar todas las páginas de memoria del proceso al disco duro. Así, se libera del nodo origen la necesidad de mantener en memoria principal los datos restantes del proceso.

Gestión de procesos distribuidos



Gestión de procesos distribuidos

- **Threads:**
- Si el proceso está subdividido en hilos, se complica aún más la migración de los procesos y la memoria:
 - Se pueden migrar hilos, manteniendo sistemas de llamadas a procedimientos remotos como formas de comunicación.
 - No se puede migrar toda la memoria si no han migrado todos los threads del proceso. Quedarán páginas del proceso distribuidas entre los nodos, la mejor opción sería enviar las páginas bajo demanda.

Gestión de procesos distribuidos

- **Ficheros abiertos:**
- Cuando se migra un proceso con ficheros abiertos, pueden darse varios casos:
 - El proceso se migra de forma temporal a un nodo, y durante ese tiempo no hace lecturas.
 - El proceso se migra, y hace lecturas/escrituras.
 - Se puede duplicar el fichero en el nodo destino
 - Problemas si otro proceso abre el mismo fichero en la máquina origen
 - Se pueden usar cachés de disco y algoritmos de coherencia de caché sobre los archivos abiertos
 - Se pueden compartir solo bloques de disco, para evitar cargar la red con datos que no sean necesarios.

Gestión de procesos distribuidos

- Ejemplo de migración: (S.O. AIX de IBM)
 - Los procesos se pueden migrar a sí mismos. Seleccionan una máquina objetivo, y envían un mensaje de petición de tarea remota. El mensaje incluye parte de la imagen del proceso e información sobre los ficheros abiertos.
 - En este momento el proceso queda suspendido, dejando el trabajo de migración a otra parte del programa, y llevando a disco el estado de la memoria,
 - En el destino, un proceso servidor del núcleo crea un hijo, pasándole la información del mensaje.

Gestión de procesos distribuidos

- Ejemplo de migración: (S.O. AIX de IBM)
 - El nuevo proceso solicita datos al origen (argumentos, información de pila, etc...) para completar su creación. Se sigue copiando las páginas del código del programa por demanda desde el sistema de ficheros global.
 - Una vez acabado el proceso de migración, envía un mensaje al nodo origen para que destruya al proceso inicial.
- Un caso similar sería que la migración la invocara un proceso externo. Los primeros pasos serían para el proceso a migrar, y llevar a disco duro compartido el estado de la memoria. A partir de ahí continuaría el mismo esquema.

Gestión de procesos distribuidos

- **Negociación de la migración:**
- Otro punto importante es la negociación de “quién se queda con el proceso”. Un ejemplo de mecanismo de negociación del sistema operativo distribuido Charlotte.
 - Se deja la responsabilidad de migración y planificación a largo plazo a la utilidad “Starter”.
 - Cada proceso Starter se encarga de vigilar el estado de un grupo de máquinas dentro del sistema. Cada cierto tiempo recopila estadísticas del kernel de cada una de las máquinas.
 - La decisión de migrar se toma al poner de acuerdo dos procesos Starter.

Gestión de procesos distribuidos

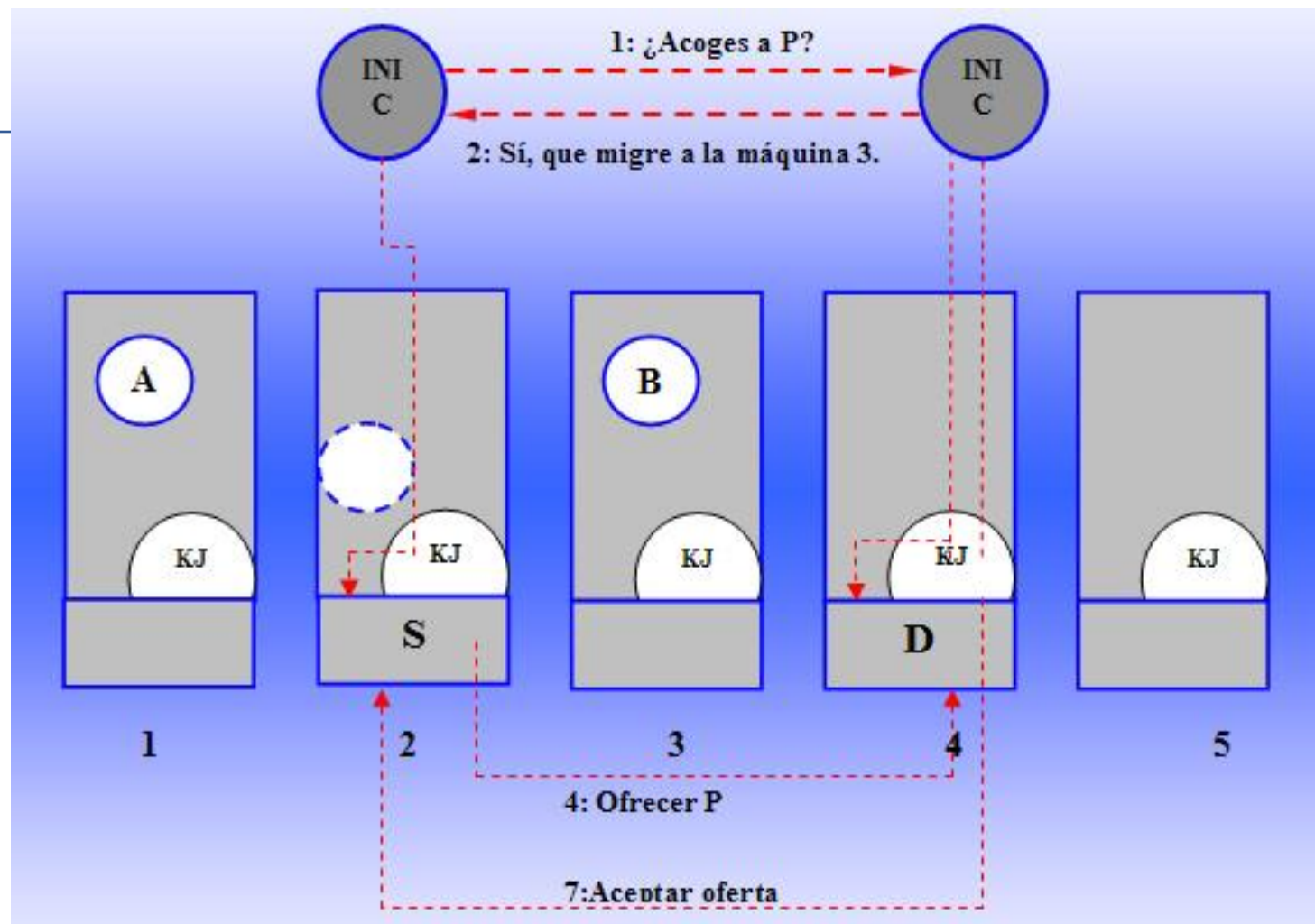
■ **Negociación de la migración:**

- **1** El Starter de una máquina origen “S” decide que un proceso “P” debe migrar a una máquina destino “D”. Primero envía un mensaje al Starter de D para solicitar una transferencia.
- **2** Si el Starter de D está listo para recibir procesos, envía un mensaje positivo.
- **3** El Starter de S comunica al kernel su decisión de migrar a P a D. El kernel le envía un mensaje de “oferta” a D con las estadísticas de P, para que decida si en el estado actual D puede hacerse cargo.

Gestión de procesos distribuidos

■ **Negociación de la migración:**

- **4** Si D está falto de recursos, puede rechazar la oferta. En caso contrario, le pasa el mensaje a su Starter, para que decida si acepta o no al proceso.
- **5** Si el Starter acepta a P, se manda un mensaje de “Inmigrar” el proceso. El kernel de D reenvía la petición a S, y reserva los recursos necesarios para recibir a P.



Gestión de procesos distribuidos

- Al igual que ocurría con los sistemas de memoria compartida, al tener varios nodos intentando acceder a un recurso compartido se necesitan sistemas para asegurar que solo uno de ellos acceda.
 - No debe ocurrir que un proceso que solicita un acceso a una sección crítica sea retardado de forma indefinida (inanición/interbloqueo).
 - Cuando la sección crítica está libre, cualquier proceso debería acceder sin retrasos.
 - No se puede suponer la velocidad relativa de los procesos o el número de procesadores.
 - Un proceso debería de permanecer en la región crítica un tiempo finito.

Gestión de procesos distribuidos

- **Exclusión mutua**
- Es muy importante el orden de envío de mensajes para evitar interbloqueos, pero al ser un sistema distribuido, podrían recibirse fuera de orden. Es necesario un sistema de ordenación:
 - Suponiendo que todos los ordenadores tienen el mismo reloj, se añaden a cada mensaje una etiqueta de tiempo que permita la reordenación de los mensajes recibidos.

Gestión de procesos distribuidos

■ Cola distribuida:

- Uno de los primeros enfoques para proporcionar la exclusión mutua en entornos distribuidos consiste en una mantener una “cola distribuida”. Se basa en las siguientes suposiciones:
 - Cada nodo contiene un proceso que realizará el acceso a los recursos en exclusión mutua. Este proceso actuará como árbitro.
 - Los mensajes se reciben en el mismo orden que se enviaron.
 - La red está completamente conectada: cualquier nodo puede enviar mensajes a otro nodo de la red.

Gestión de procesos distribuidos

- **Cola distribuida:**

- Todos los nodos mantienen un vector de estado del recurso compartido, que funciona como una cola FIFO.

- **Proceso:**

- **1** El nodo P_i necesita acceder a un recurso compartido. Encola en su vector local la petición, y envía un mensaje de petición a todos los nodos de la red, indicando la posición local que tendría dentro del vector.
- **2** El resto de procesos reciben el mensaje para encolar al nodo P_i . Si en la posición indicada no hay ningún otro proceso, se contesta a P_i aceptando la petición.

Gestión de procesos distribuidos

■ Cola distribuida:

- **3** Cuando el proceso P_i se encuentra a la salida de la cola, puede acceder a la sección de exclusión mutua.
- **4** Cuando acaba con la sección, envía un mensaje para que el resto de nodos le saquen del vector FIFO.
- **5** El resto de procesos recibirán el mensaje, y comprobarán si es su turno para acceder a la sección crítica.

Virtualización de sistemas

- Permite ejecutar varias instancias de un SO o aplicaciones simultáneamente
 - Transparencia al SO (o casi)
 - Extensiones de Hardware
- Tecnología “novedosa”
 - Originaria de desarrollos de IBM en los años 60 (SIMMON y CP/CMS).
 - Particionamiento lógico de mainframes.
- Desarrollos más modernos (tecnología x86)
 - Connectix
 - VMware

Motivos para la virtualización

- Baja utilización de la infraestructura
 - Del orden del 10-15 % por servidor
- Incrementos de los costes de la infraestructura física
 - Inversión en equipos, mantenimiento, consumo, etc.
- Incremento en los costes de personal de TI
 - Mayor infraestructura → Más personal y más cualificado
- Protección frente a contingencias insuficiente
 - Coste mayor cuanto más grande sea la organización
- Mantenimiento elevado de los escritorios de usuario
 - Difícil garantizar seguridad sin afectar a su productividad

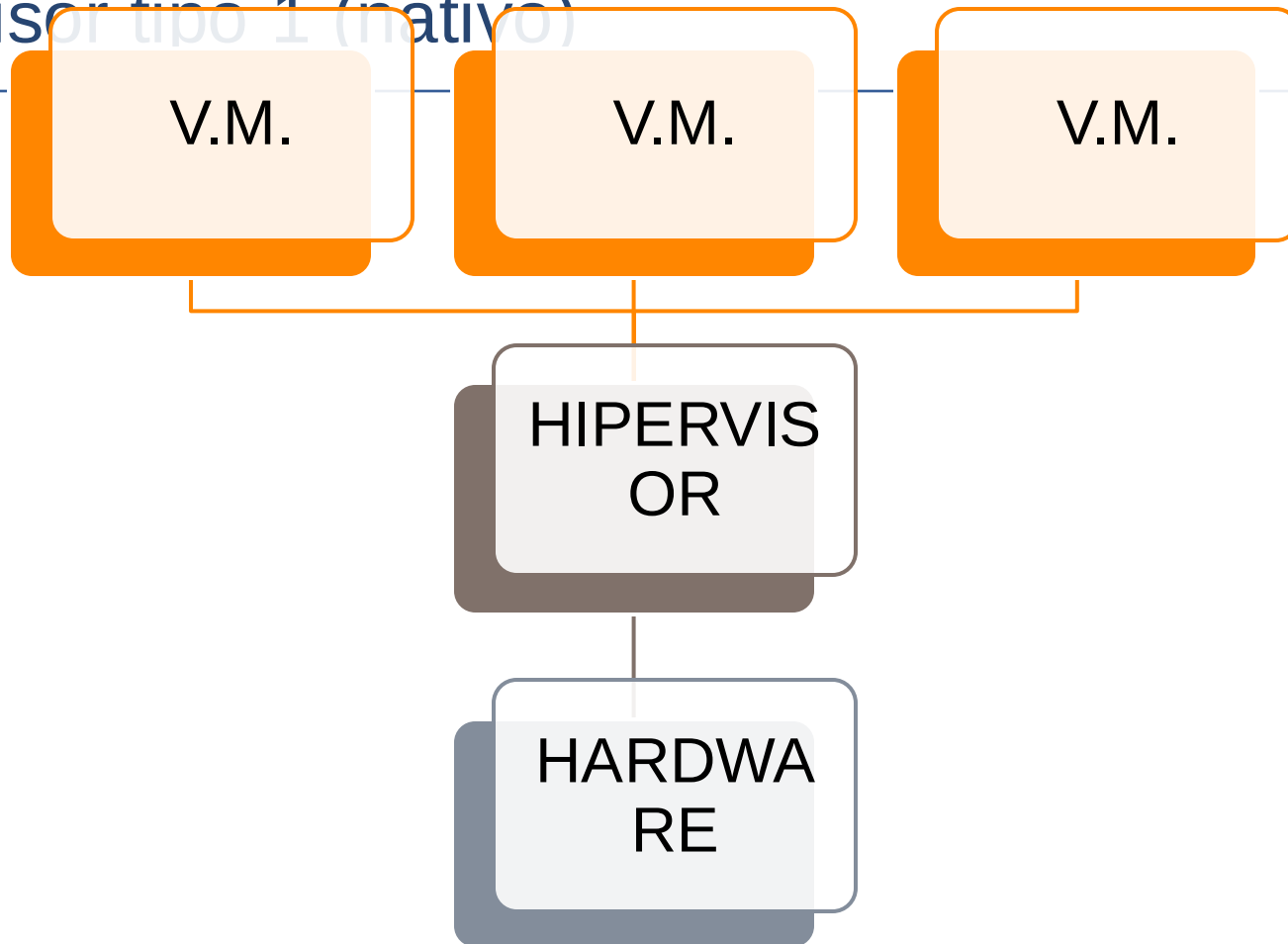
Ventajas de la virtualización

- Menor inversión en infraestructura
- Menor coste de mantenimiento
- Mayor flexibilidad y aprovechamiento de los recursos
- Mayor seguridad y tolerancia a fallos
- Seguro que muchas más...

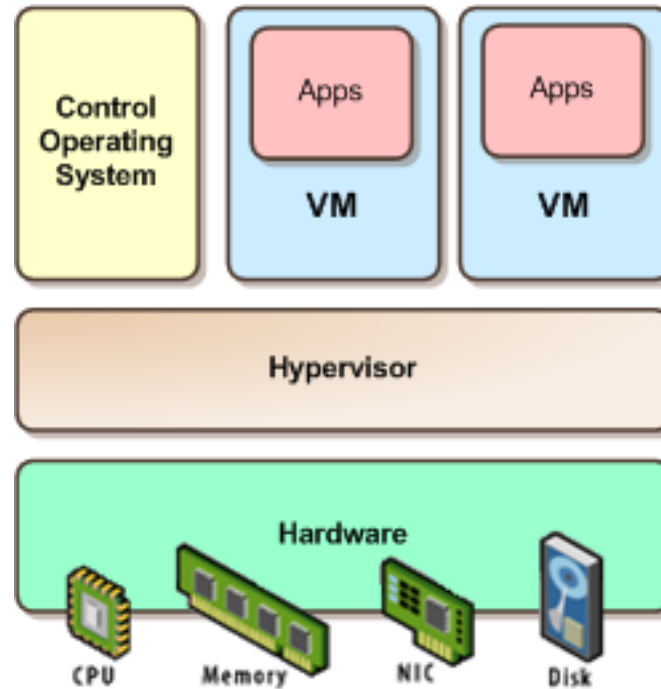
Hipervisor/VMM

- Pieza software que se encarga de proporcionar los mecanismos específicos de virtualización.
 - Tipo 1 (nativo, bare metal): Directamente sobre el hardware. Ej: Amazon EC2 (Xen), VMWare ESX/ESXi
 - Tipo 2 (hosted): Aplicación que ejecuta dentro del SO. Ej: VMWare WorkStation
- También llamado Virtual Machine Manager (VMM).

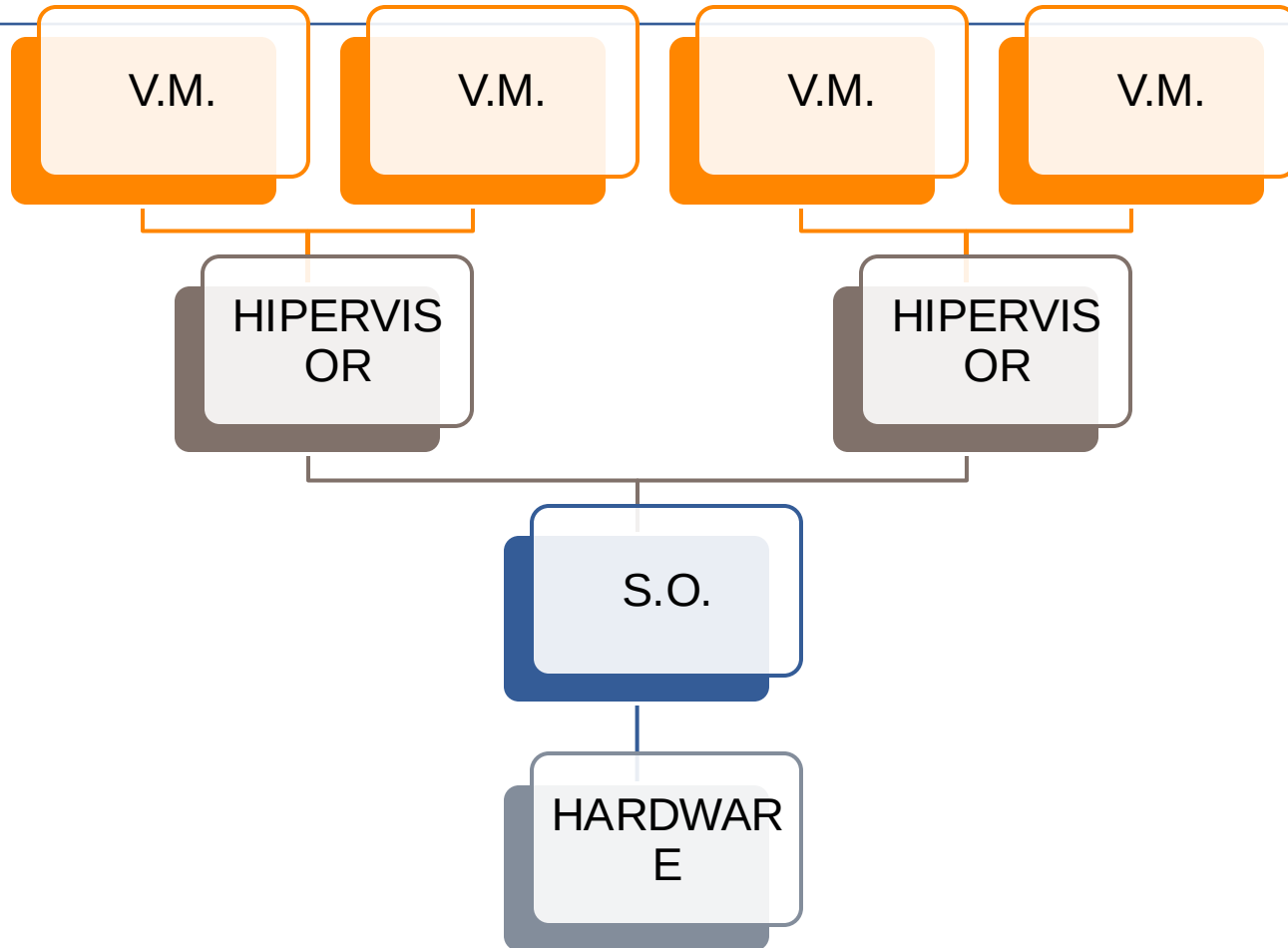
Hipervisor tipo 1 (nativo)



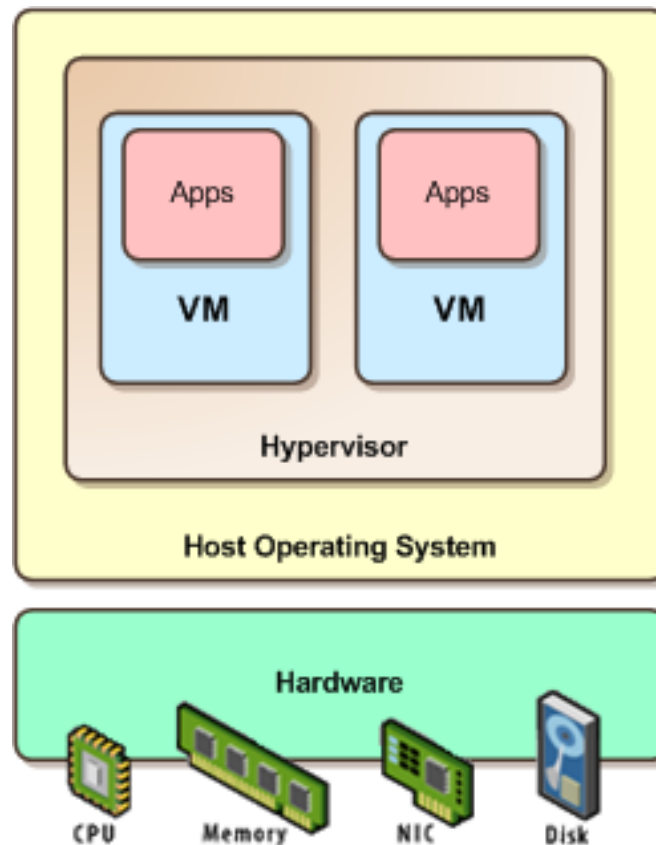
Hipervisor tipo 1 (nativo)



Hipervisor tipo 2 (hosted)



Hipervisor tipo 2 (hosted)



Tipos de virtualización

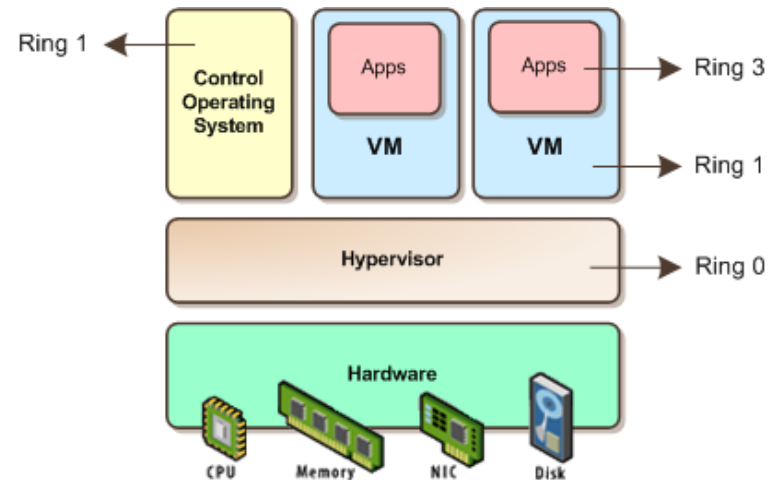
- Distintas estrategias enfocadas a escenarios diferentes
 - Mayor o menor heterogeneidad
 - Rendimiento vs. Mantenimiento
- Tipos de virtualización:
 - Virtualización completa (full)
 - Virtualización asistida por Hardware
 - Paravirtualización
 - Virtualización a nivel de Sistema Operativo

Virtualización completa (full)

- Abstracción completa del hardware subyacente.
- Proporciona soporte para SOs sin modificar.
- El VMM proporciona emulación de CPU para gestionar operaciones privilegiadas.
 - Comprueba las instrucciones que se van a ejecutar
 - Si es posible, las ejecuta directamente
 - Si no, las traduce a unas que tengan un efecto idéntico en el HW virtualizado
 - En el caso de x86, hay algunas instrucciones no virtualizables
- El VMM requiere a la vez recursos del sistema y tiempo de ejecución. Esto implica un menor rendimiento que otras alternativas como paravirtualización

Virtualización completa (full)

- Vmware
- Parallels
- VirtualBox
- Win4BSD
- QEmu
- Oracle VM
- etc.

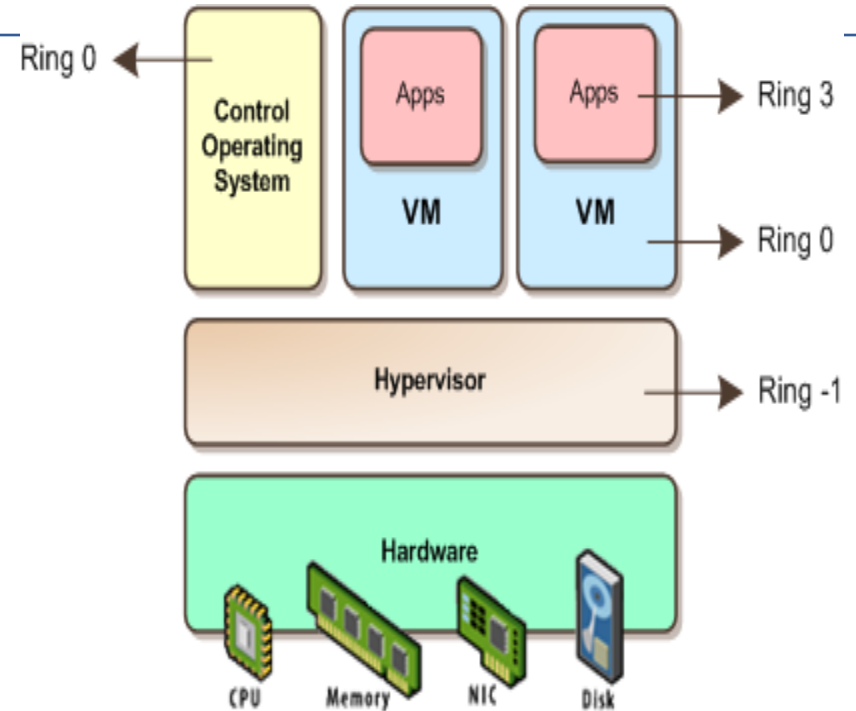


Virtualización asistida por HW

- Muy parecido al anterior
- Diferencia:
 - La captura de instrucciones se lleva a cabo con los mecanismos HW presentes en el procesador
 - Menor sobrecarga
- Específico de procesadores modernos
 - Intel VT
 - AMD-V
 - Itanium VT-i (también de Intel)

Virtualización asistida por HW

- VMware Workstation
- Xen
- Virtual Box
- KVM
- MS Hyper-V



Paravirtualización

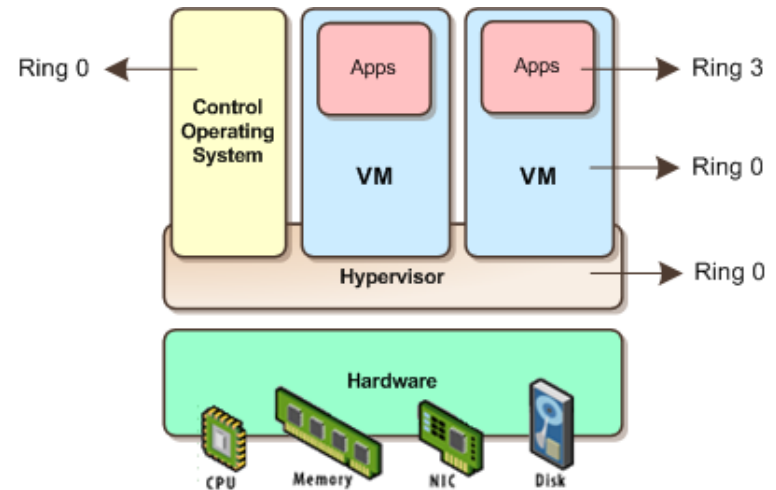
- El SO es consciente de estar ejecutando en un entorno virtualizado.
 - Necesitan ser modificados específicamente para ejecutar en el VMM
- Se comunica con un hipervisor/VMM de tipo 1 para llevar a cabo las operaciones no virtualizables.
 - Reemplaza las operaciones privilegiadas por hiperllamadas
- Arquitectura habitual:
 - El hipervisor dialoga con el hardware (nivel más bajo).
 - Instancia de SO privilegiada comunicando con el hipervisor.
 - Resto de instancias gestionadas por la instancia privilegiada.

Paravirtualización

- Ventaja: rendimiento.
- Inconveniente: Suele limitar el soporte a sistemas operativos libres como Linux o de fabricantes que desarrollan sus modificaciones
 - Virtual Machine Interface (VMI)
 - Mecanismo de comunicación estándar entre el SO y el Hipervisor propuesto por VMware.
 - Permite que el SO pueda ejecutar tanto en una máquina virtual como en una real.

Paravirtualización

- Xen
- VMware ESXi
- TRANGO
- Sun's Logical Domains
- IBM's z/VM
- Virtual Box



Virtualización a nivel de SO

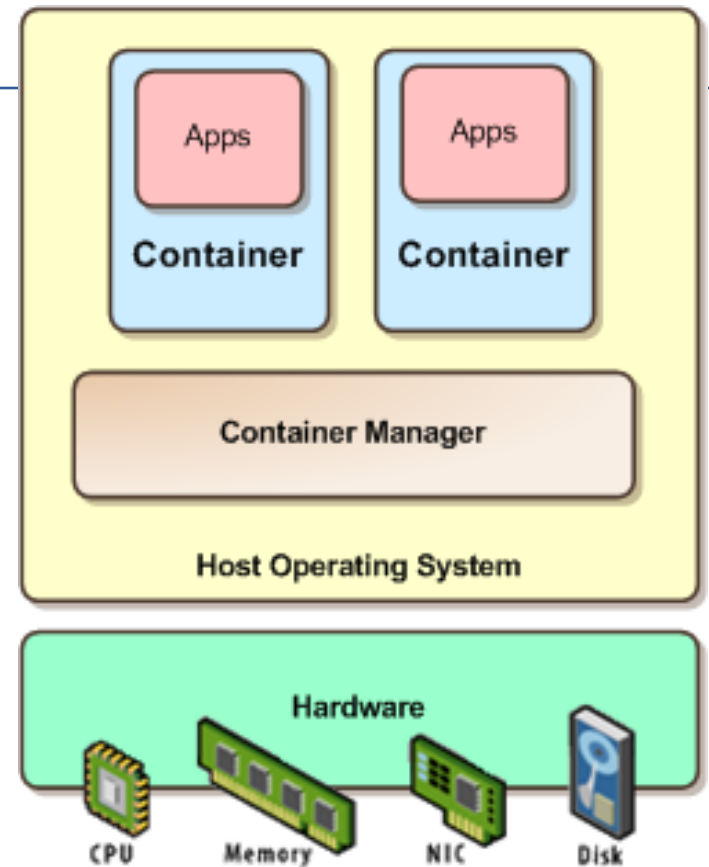
- Permite la ejecución de múltiples instancias de un mismo SO.
 - Pueden ser distintas “versiones” del mismo SO (e.g. Linux).
 - A estas instancias se les llama containers o jails.
 - Una especie de chroot avanzado (Unix).
- De uso en entornos compartidos (múltiples servicios, hostings, etc.)
 - Balanceo de carga.

Virtualización a nivel de SO

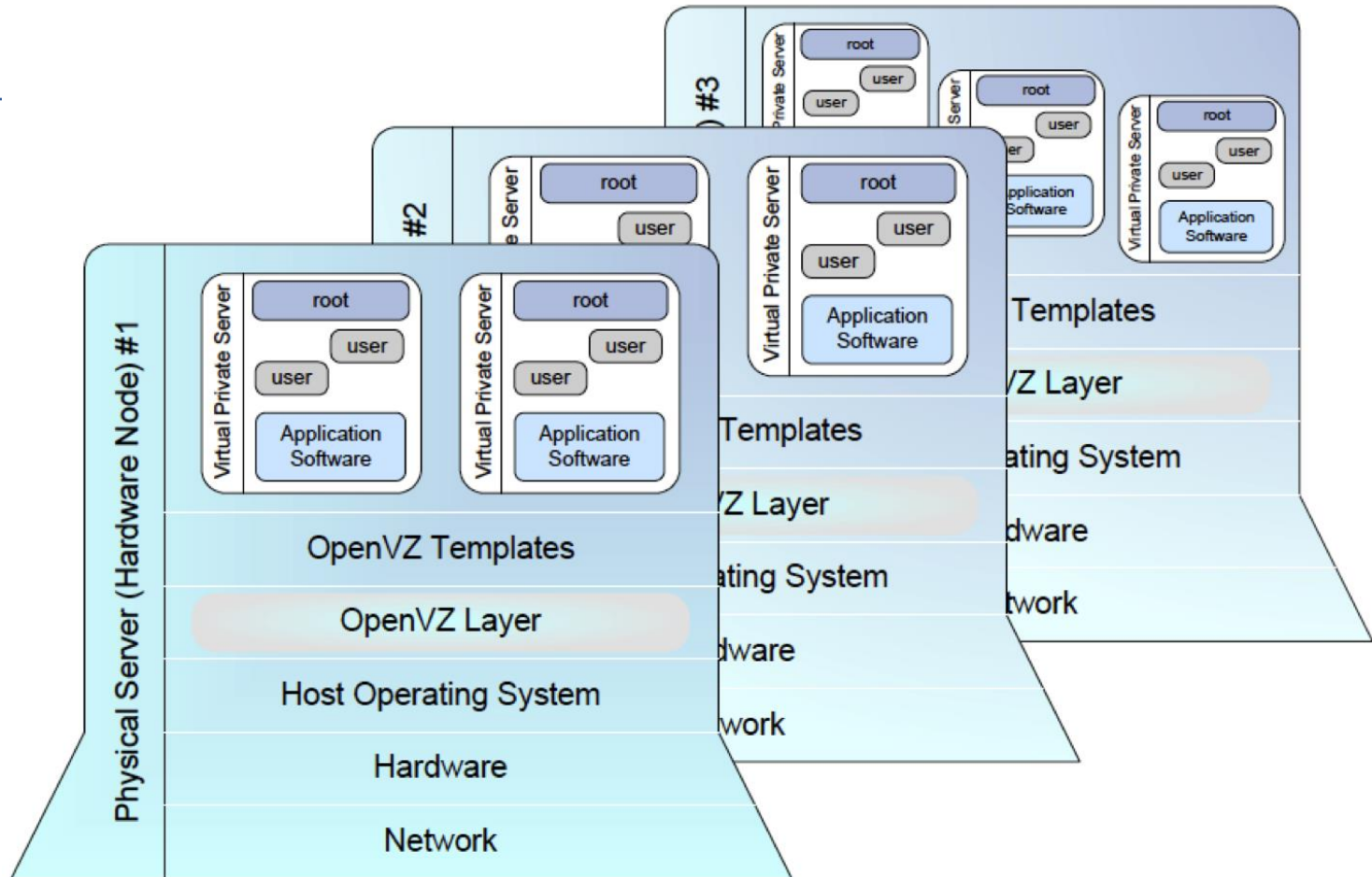
- Instancias sin modificar.
 - Pequeño impacto en el rendimiento del conjunto.
- Poca flexibilidad.
- Sistemas de almacenamiento con copy-on-write.
 - Las instancias comparten sistema de ficheros.
 - Se crea una nueva copia del fichero cuando otra instancia lo modifica.

Virtualización a nivel de SO

- OpenVZ
- Parallels Virtuozzo Containers
- Solaris Containers
- FreeBSD Jail
- Linux-VServer



Virtual



Preguntas, dudas...

